PROJECT REPORT

ATM Withdrawal Program

Using Python

Subject: Python Programming

Level: Second-Year B.Tech Engineering

Focus: Basic Python, exception handling and Tkinter

Submitted by

Kartik Kale
Roll Number: 03
PRN: SOE25201020081

Rutika Budhe
Roll Number: 19
PRN: SOE25201020034

A simple graphical demonstration of PIN verification, balance enquiry, withdrawal and deposit operations.

01 / OVERVIEW

Abstract

The ATM Withdrawal Program is a small desktop application developed in Python using Tkinter. It demonstrates how basic banking operations can be represented through variables, functions, conditional statements and a graphical interface. The project is intended for a Second-Year B.Tech Python Programming mini-project, with particular emphasis on exception handling.

A user first enters the demonstration PIN, 1234. After successful verification, the user can check an initial balance of ₹10,000, withdraw money or deposit money. Each successful transaction updates the balance held in memory and displays an explanatory message. The Exit button closes the application.

The main learning outcome is safe handling of invalid input. Empty fields, incorrect PINs, text or fractional amounts, zero and negative amounts, and withdrawals exceeding the available balance are handled using try-except blocks and ValueError. These situations produce readable messages instead of terminating the program. Failed transactions do not change the balance.

The implementation is deliberately limited to one application file and uses the Python standard library. Amounts are stored as integers in whole rupees. No database, API, external application package or real banking connection is needed. The balance is reset whenever the program restarts.

Validation summary

The actual Tkinter widgets and their callbacks passed 25 checks on Windows with Python 3.12 and Tk 8.6. Genuine screenshots document the running application. The complete code, test results and repository address are included in this report.

02 / CONTENTS

Index

Cover page	1
Abstract	2
Index	3
Introduction	4
Problem statement and objectives	5
Methodology and flowchart	6
Python concepts used	7
Source code: validation and operations	8
Source code: graphical interface	9
Output screenshots: login and dashboard	10
Output screenshots: withdrawal and errors	11
Testing and conclusion	12
GitHub repository and execution	13

03 / BACKGROUND

Introduction

An Automated Teller Machine (ATM) provides a convenient interface for common banking services such as withdrawing money and checking an account balance. A real ATM verifies a customer and communicates with banking infrastructure before completing a transaction.

This project models only the basic sequence of entering a PIN, choosing an operation, validating an amount and updating a balance. The interface runs locally as a Tkinter window. It does not connect to a bank or dispense money.

Why input validation matters

A program cannot assume that every entry is valid. A user may leave the amount blank, type letters, enter a negative number or request more money than is available. If these cases are ignored, the program can crash or produce an incorrect balance.

Exception handling provides a clear way to respond to these problems. The program checks inputs before modifying the balance, raises ValueError when a rule is violated, and catches the error to display a helpful message. The user can then correct the entry and try again.

Scope of the demonstration

• One demonstration PIN and one balance stored in memory.

• Whole-rupee withdrawals and deposits through a basic GUI.

• No account registration, persistent transaction history or database.

• A fixed demonstration PIN is a teaching aid; this project is not a production banking or authentication system.

04 / PROJECT DEFINITION

Problem & objectives

Problem statement

Develop a simple Python desktop application that verifies a PIN and supports balance enquiry, withdrawal and deposit operations. The application must reject invalid input safely, preserve the balance when a transaction fails, and give the user meaningful feedback.

Objectives

• Verify the entered PIN against the demonstration PIN 1234.

• Display an initial balance of ₹10,000 after login.

• Accept positive whole-rupee withdrawals only when sufficient funds exist.

• Accept positive whole-rupee deposits and update the balance.

• Use try-except and ValueError to handle invalid entries without crashing.

• Provide clear messages and an Exit button in a readable Tkinter interface.

• Keep the code short enough to understand and explain during a viva.

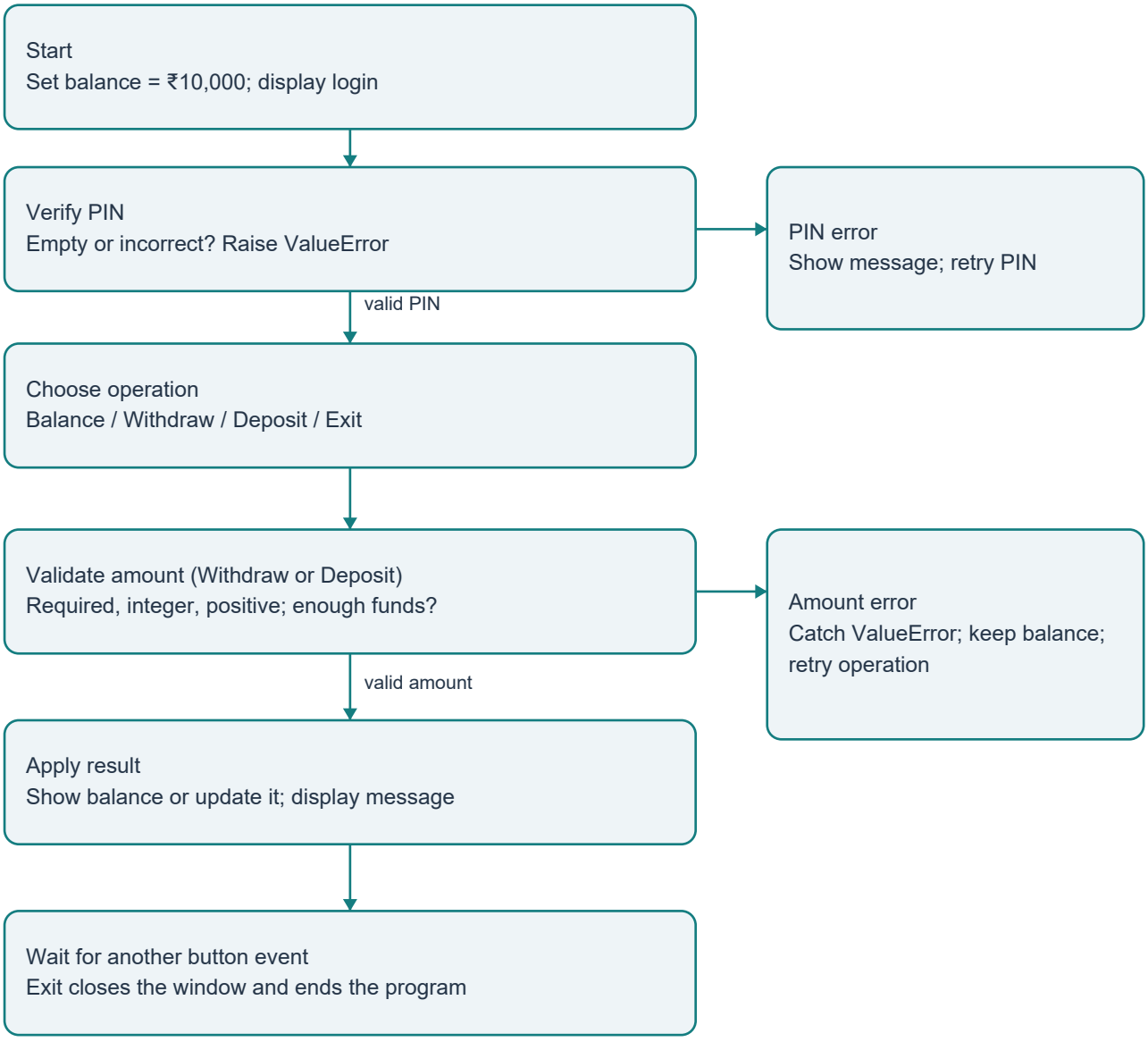
Expected behaviour

A valid withdrawal of ₹2,500 reduces ₹10,000 to ₹7,500. A request to withdraw ₹8,000 from ₹7,500 is rejected and leaves the balance unchanged. A later deposit of ₹1,500 increases ₹7,500 to ₹9,000.

05 / WORKING AND FLOWCHART

Methodology

Tkinter waits for button events. Login enables the account controls. Each transaction validates the input before updating the shared balance. The same message area displays success or a caught ValueError.



Balance enquiry needs no amount. Exit is available before and after login. Error branches return control to the Tkinter event loop so the user can retry.

06 / IMPLEMENTATION NOTES

Python concepts used

Variables and data types

PIN is a string, balance and amount are integers, and logged_in is a Boolean. The entry widgets return strings, which are stripped of surrounding whitespace before validation.

Functions

login() verifies the PIN; check_balance() displays funds; transact() handles withdrawals and deposits; show_message() presents feedback; create_app() builds and returns the window.

Conditional statements

if statements reject empty input, wrong PINs, non-positive amounts and insufficient funds. The operation determines whether the program subtracts from or adds to the balance.

Exception handling

try encloses the validation and transaction steps. int(text) can raise ValueError for a non-integer. Explicit raise ValueError statements represent invalid application inputs. except ValueError catches these errors and displays a readable message.

Tkinter GUI and callbacks

Labels, entry fields, buttons, frames and a separator form the interface. Button commands call Python functions. mainloop() processes user events, and root.destroy closes the window.

Lists, loop and shared state

A short list and for loop create the three account buttons. account_widgets allows their states to change together. global makes the simple shared balance and login status available to callbacks.

07 / VALIDATION AND OPERATIONS

Source code

```
"""A simple ATM demonstration using Tkinter and ValueError."""
import tkinter as tk
from tkinter import ttk

PIN = "1234"
balance = 10000  # Whole rupees; resets when the program restarts.
logged_in = False


def show_message(text, error=False):
    message.config(text=text, fg="#b42318" if error else "#176447")


def login():
    global logged_in
    try:
        entered_pin = pin_entry.get().strip()
        if not entered_pin:
            raise ValueError("Please enter your PIN.")
        if entered_pin != PIN:
            raise ValueError("Invalid PIN. Please try again.")
        logged_in = True
        for widget in account_widgets:
            widget.config(state="normal")
        pin_entry.delete(0, tk.END)
        pin_entry.config(state="disabled")
        login_button.config(state="disabled")
        amount_entry.focus_set()
        show_message("Login successful. Choose an operation.")
    except ValueError as error:
        show_message(str(error), error=True)


def check_balance():
    if logged_in:
        show_message(f"Available balance: ₹{balance:,}")
    else:
        show_message("Please log in first.", error=True)


def transact(operation):
    global balance
    try:
        if not logged_in:
            raise ValueError("Please log in first.")
        text = amount_entry.get().strip()
        if not text:
            raise ValueError("Please enter an amount.")
        try:
            amount = int(text)
        except ValueError:
            raise ValueError("Enter a whole number of rupees.")
        if amount <= 0:
            raise ValueError("Amount must be greater than zero.")
        if operation == "Withdraw":
            if amount > balance:
                raise ValueError("Insufficient balance.")
            balance -= amount
            result = "Withdrawal successful"
        else:
            balance += amount
            result = "Deposit successful"
        show_message(f"{result}: ₹{amount:,}\n"
                     f"Available balance: ₹{balance:,}")
        amount_entry.delete(0, tk.END)
    except ValueError as error:
        show_message(str(error), error=True)
```

07 / GRAPHICAL INTERFACE

Source code

```
68
69
70  def create_app():
71      global pin_entry, amount_entry, login_button, message
72      global account_widgets
73      root = tk.Tk()
74      root.title("ATM Withdrawal Program")
75      root.geometry("510x530")
76      root.resizable(False, False)
77      root.configure(bg="#f3f6fa")
78      root.option_add("*Font", ("Segoe UI", 11))
79      frame = ttk.Frame(root, padding=26)
80      frame.pack(fill="both", expand=True, padx=14, pady=14)
81      ttk.Label(frame, text="ATM WITHDRAWAL PROGRAM",
82                font=("Segoe UI", 17, "bold")).pack(pady=(0, 6))
83      ttk.Label(frame, text="Python Programming | Mini-project").pack()
84      ttk.Separator(frame).pack(fill="x", pady=18)
85      ttk.Label(frame, text="Enter PIN").pack(anchor="w")
86      pin_entry = ttk.Entry(frame, show="*", justify="center")
87      pin_entry.pack(fill="x", pady=(5, 8))
88      login_button = ttk.Button(frame, text="Login", command=login)
89      login_button.pack(fill="x")
90      ttk.Label(frame, text="Amount (whole rupees)").pack(
91          anchor="w", pady=(16, 5))
92      amount_entry = ttk.Entry(frame, state="disabled")
93      amount_entry.pack(fill="x")
94      actions = ttk.Frame(frame)
95      actions.pack(fill="x", pady=12)
96      account_widgets = [amount_entry]
97      commands = [("Check Balance", check_balance),
98                  ("Withdraw", lambda: transact("Withdraw")),
99                  ("Deposit", lambda: transact("Deposit"))]
100     for column, (label, command) in enumerate(commands):
101         button = ttk.Button(actions, text=label, command=command,
102                             state="disabled")
103         button.grid(row=0, column=column, padx=3, sticky="ew")
104         actions.columnconfigure(column, weight=1)
105         account_widgets.append(button)
106     message = tk.Label(frame, text="Welcome! Please log in.",
107                        wraplength=400, height=3, bg="#e8eef5",
108                        fg="#334155", font=("Segoe UI", 11))
109     message.pack(fill="x", pady=(0, 12))
110     ttk.Button(frame, text="Exit", command=root.destroy).pack(fill="x")
111     ttk.Label(frame, text="Demonstration only | No real transactions",
112               font=("Segoe UI", 9)).pack(pady=(12, 0))
113     pin_entry.focus_set()
114     return root
115
116
117 if __name__ == "__main__":
118     create_app().mainloop()
```

08 / LOGIN AND DASHBOARD

Output screenshots

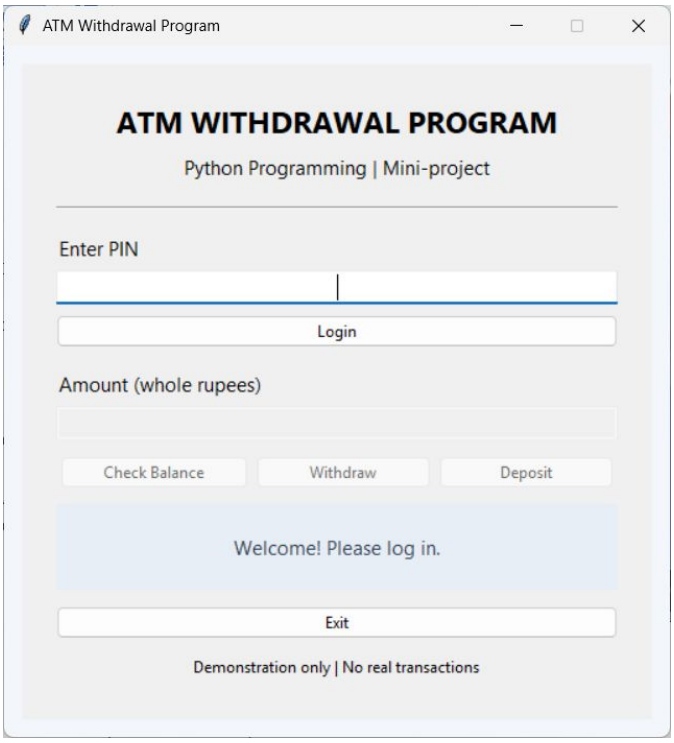


Figure 1. Login screen. The actual application opens with the account controls disabled until PIN verification succeeds.

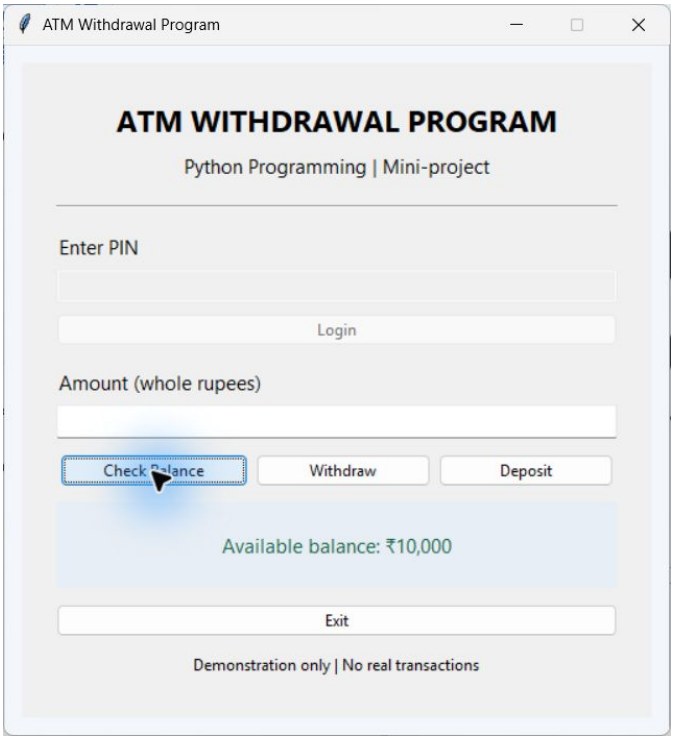


Figure 2. Dashboard. After login with PIN 1234, Check Balance displays the starting balance of ₹10,000.

08 / WITHDRAWAL AND ERRORS

Output screenshots

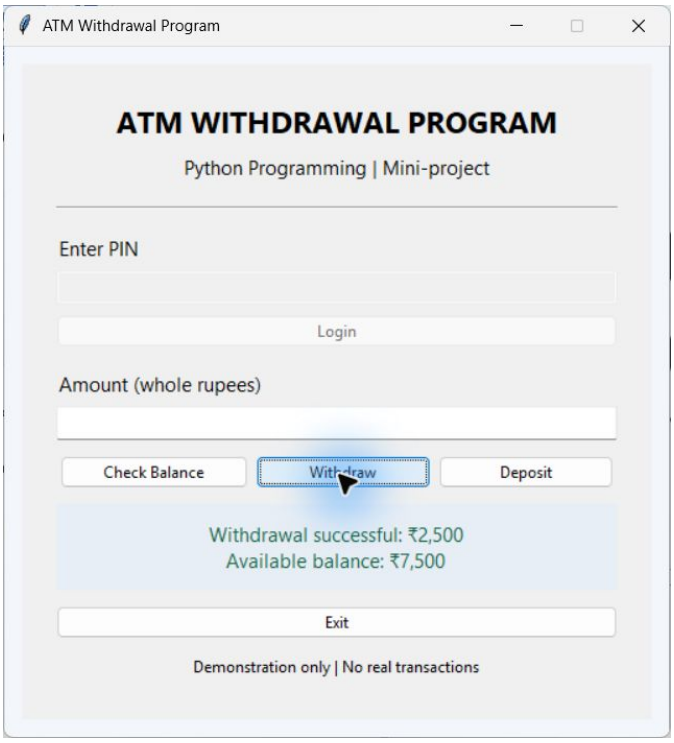


Figure 3. Successful withdrawal. Withdrawing ₹2,500 from ₹10,000 displays an updated balance of ₹7,500.

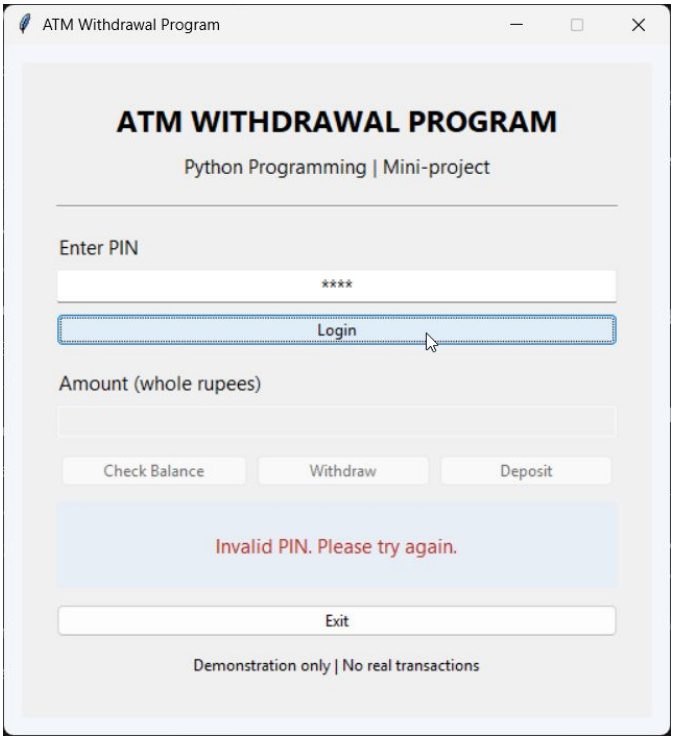


Figure 4. Error handling. An incorrect PIN produces a clear error message while the account controls stay disabled.

09 / ACTUAL RESULTS

Testing & conclusion

Executed on Windows with Python 3.12 and Tk 8.6. The verification script created real Tkinter widgets and invoked their button callbacks without mocks. All 25 checks passed; the grouped results are below.

Test inputs / action	Observed result	Checks
Before login	Account controls disabled	1 PASS
PIN: empty, spaces, 9999, abcd	Missing/invalid PIN errors; no login	4 PASS
PIN: 1234	Login succeeds; controls enabled	1 PASS
Check Balance	Initial balance: ₹10,000	1 PASS
Withdraw ₹2,500	Balance becomes ₹7,500	1 PASS
Withdraw ₹8,000 from ₹7,500	Insufficient balance; no change	1 PASS
Withdraw and deposit: empty, spaces	Please enter an amount; no change	4 PASS
Withdraw and deposit: abc, 12.5	Whole-number error; no change	4 PASS
Withdraw and deposit: 0, -500	Positive-amount error; no change	4 PASS
Deposit ₹1,500	Balance becomes ₹9,000	1 PASS
Withdraw exactly ₹9,000	Balance becomes ₹0	1 PASS
Withdraw ₹1 from ₹0	Insufficient balance; remains ₹0	1 PASS
Exit button	Tk window destroyed	1 PASS

Conclusion

The program meets the project objectives using a small Tkinter interface and standard Python features. It validates input, handles errors without crashing and updates balances correctly. The project demonstrates how exception handling improves the reliability of a simple interactive application.

10 / SOURCE AND EXECUTION

GitHub repository

Public project repository

https://github.com/niiharbute/ATM-Withdrawal-Program-Python

The repository contains main.py, README.md, this PDF report and four genuine screenshots in the screenshots folder. The application uses only tkinter and tkinter.ttk from the Python standard library.

How to run

1. Install Python 3 with Tkinter support and use a graphical desktop.

2. Download or clone the repository and open its folder in a terminal.

3. Run `python main.py`.

4. Enter PIN 1234, then click Login.

5. Check the balance, or enter a positive whole-rupee amount and choose Withdraw or Deposit. Click Exit to close.

Example demonstration

Login → Check Balance (₹10,000) → Withdraw ₹2,500 (₹7,500) → attempt to withdraw ₹8,000 (error, balance unchanged) → Deposit ₹1,500 (₹9,000) → Exit.

Limitations

The fixed PIN and in-memory balance are for teaching purposes. The application has no persistence, multi-user accounts, real bank connection or fractional-rupee support. Restarting it restores the initial balance of ₹10,000.